

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

КУРСОВАЯ РАБОТА

**на тему: «Применение рекуррентных нейронных сетей
в обработке естественного языка»**

по дисциплине «Нейрокомпьютерные системы»

Обучающегося 1 курса группы 12-25РПм Направления подготовки 09.04.04
Трифопова Д.С.

Научный руководитель:
Круглый Д.Г.

Оглавление

ВВЕДЕНИЕ	2
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ	4
1.1 Основы обработки естественного языка	4
1.2 Нейронные сети и их применение	5
2 РЕКУРРЕНТНЫЕ НЕЙРОННЫЕ СЕТИ	7
2.1 Архитектура RNN	7
2.2 Проблема исчезающего градиента	8
2.3 Модификации RNN: LSTM и GRU	10
2.3.1 Архитектура LSTM	10
2.3.2 Архитектура GRU	11
2.3.3 Сравнение LSTM и GRU	11
3 ПРИМЕНЕНИЕ RNN В ЗАДАЧАХ NLP	12
3.1 Классификация текстов	12
3.1.1 Архитектура RNN для классификации	12
3.1.2 Преимущества использования RNN	13
3.1.3 Современные подходы	13
3.1.4 Применение на практике	13
3.2 Машинный перевод	14
3.2.1 Архитектура Encoder-Decoder	14
3.2.2 Двухнаправленные кодировщики (Bi-directional Encoder)	14
3.2.3 Механизм внимания (Attention)	15
3.2.4 Практические применения	15
3.2.5 Преимущества RNN в машинном переводе	15
3.3 Генерация текста	16
3.3.1 Использование RNN для генерации текста	16
3.3.2 Роль LSTM и GRU	16
3.3.3 Стратегии генерации текста	17
3.3.4 Применение генерации текста	17
3.3.5 Преимущества и ограничения	18
ЗАКЛЮЧЕНИЕ	19
Список использованных источников	21

ВВЕДЕНИЕ

Современный этап развития информационных технологий характеризуется стремительным ростом объема текстовой информации. Электронные документы, социальные сети, новостные порталы, системы электронной коммерции и корпоративные базы данных ежедневно генерируют огромные массивы текстовых данных. Эффективная обработка, анализ и интерпретация этой информации становятся важнейшими задачами современной вычислительной техники.

Обработка естественного языка (Natural Language Processing, NLP) представляет собой направление искусственного интеллекта, объединяющее методы компьютерной лингвистики, машинного обучения и математической статистики [3]. Основной целью NLP является разработка алгоритмов, позволяющих автоматизировать анализ, понимание и генерацию текстов на естественных языках. Данная область активно развивается благодаря увеличению вычислительных мощностей и появлению новых методов машинного обучения.

Одной из ключевых проблем обработки текстовых данных является их последовательная природа. В отличие от структурированных данных, текст представляет собой последовательность слов, смысл которых зависит от контекста. Для корректной интерпретации предложения необходимо учитывать взаимосвязь между его элементами, а также долгосрочные зависимости внутри текста. Традиционные алгоритмы обработки данных не способны эффективно решать подобные задачи без использования специализированных моделей.

Существенный вклад в развитие методов анализа последовательностей внесли искусственные нейронные сети, в частности рекуррентные нейронные сети (Recurrent Neural Networks, RNN) [3]. Данные модели обладают механизмом внутреннего состояния, позволяющим учитывать информацию о предыдущих элементах последовательности при обработке текущего входа. Это делает их особенно эффективными в задачах анализа текста, машинного перевода, распознавания речи и генерации последовательностей.

Однако классические рекуррентные нейронные сети сталкиваются с рядом ограничений, включая проблему исчезающего и взрывающегося градиента, что затрудняет обучение на длинных последовательностях. Для преодоления этих ограничений были разработаны усовершенствованные архитектуры, такие как Long Short-Term Memory (LSTM) и Gated Recurrent Unit (GRU) [3], позволяющие эффективно моделировать долгосрочные зависимости.

Актуальность выбранной темы обусловлена широким применением рекуррентных нейронных сетей в современных интеллектуальных системах, а также необходимостью изучения их теоретических основ и практических возможностей в области обработки естественного языка.

Целью данной курсовой работы является исследование принципов работы рекуррентных нейронных сетей и анализ их применения в задачах обработки естественного языка.

Для достижения поставленной цели необходимо решить следующие задачи:

- Рассмотреть основные понятия и методы обработки естественного языка;
- Изучить математические и архитектурные основы рекуррентных нейронных сетей;
- Проанализировать модификации RNN (LSTM, GRU) и их особенности;
- Исследовать области практического применения RNN в задачах NLP;
- Сформулировать выводы по результатам проведенного анализа.

Объект исследования — методы и алгоритмы обработки текстовой информации.

Предмет исследования — применение рекуррентных нейронных сетей в задачах обработки естественного языка.

Практическая значимость работы заключается в систематизации теоретических сведений о рекуррентных нейронных сетях и возможности их дальнейшего использования при разработке интеллектуальных информационных систем.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

1.1 Основы обработки естественного языка

Обработка естественного языка (Natural Language Processing, NLP) представляет собой междисциплинарную область исследований, находящуюся на стыке информатики, лингвистики и искусственного интеллекта [3]. Основной задачей NLP является разработка алгоритмов и моделей, способных анализировать, интерпретировать и генерировать текстовую информацию на естественных языках.

Исторически первые системы обработки текста основывались на правилах и лингвистических моделях, сформированных экспертами. Такие системы требовали ручного описания грамматических конструкций и словарей, что делало их сложными в разработке и масштабировании. С развитием вычислительной техники и методов машинного обучения акцент сместился в сторону статистических моделей, способных автоматически извлекать закономерности из больших объёмов данных.

Обработка естественного языка включает несколько уровней анализа. На морфологическом уровне выполняется разбиение текста на отдельные единицы — токены, а также определение начальной формы слов (лемматизация) или их основы (стемминг). На синтаксическом уровне осуществляется анализ структуры предложения и выявление грамматических связей между словами. Семантический анализ направлен на выявление смысла текста и интерпретацию значений слов в контексте. Более высокий уровень — прагматический анализ — учитывает цели высказывания и контекст использования языка.

К основным задачам NLP относятся [2]:

- токенизация текста;
- лемматизация и стемминг;
- синтаксический анализ;
- классификация текстов;
- машинный перевод;
- анализ тональности;
- извлечение именованных сущностей;
- автоматическое реферирование.

Современные методы обработки естественного языка базируются преимущественно на алгоритмах машинного обучения. Особое распространение получили нейронные сети,

способные эффективно работать с большими объёмами текстовых данных. В отличие от традиционных подходов, основанных на фиксированных признаках, нейросетевые модели способны самостоятельно формировать внутренние представления текста.

Важной особенностью текстовых данных является их последовательная структура. Значение слова определяется не только его собственной формой, но и контекстом, в котором оно используется. Одно и то же слово может иметь различные значения в зависимости от окружающих его элементов. Это требует применения моделей, способных учитывать зависимость между элементами последовательности.

Таким образом, обработка естественного языка представляет собой сложную многокомпонентную задачу, требующую использования методов, способных учитывать как структуру языка, так и контекстуальные зависимости. Одним из наиболее эффективных инструментов решения подобных задач являются рекуррентные нейронные сети, которые рассматриваются в последующих разделах работы.

1.2 Нейронные сети и их применение

Искусственные нейронные сети представляют собой класс моделей машинного обучения, вдохновлённых принципами функционирования биологических нейронных систем. Основным структурным элементом нейронной сети является искусственный нейрон — вычислительный узел, принимающий входные сигналы, взвешивающий их и формирующий выходное значение посредством функции активации.

Математически работу нейрона можно описать следующим выражением:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right),$$

где x_i — входные сигналы, w_i — соответствующие весовые коэффициенты, b — смещение (bias), а f — функция активации [3]. В качестве функций активации часто используются сигмоидальная функция, гиперболический тангенс и функция ReLU.

Нейронные сети состоят из нескольких слоёв: входного, одного или нескольких скрытых и выходного слоя. Процесс обработки информации осуществляется последовательно от входа к выходу. В полносвязных (feedforward) нейронных сетях каждый нейрон текущего слоя связан со всеми нейронами предыдущего слоя.

Обучение нейронной сети заключается в подборе весовых коэффициентов таким образом, чтобы минимизировать функцию потерь. Для этого используется алгоритм обратного распространения ошибки (backpropagation) в сочетании с методами градиентного спуска [3]. В ходе обучения сеть постепенно корректирует веса, уменьшая расхождение между предсказанным и фактическим значением.

С развитием глубокого обучения (deep learning) получили распространение многослойные нейронные сети, способные автоматически извлекать сложные признаки из

исходных данных. В задачах обработки изображений широко применяются сверточные нейронные сети (CNN), тогда как для анализа последовательных данных, включая текст и речь, используются рекуррентные нейронные сети.

Применение нейронных сетей в обработке естественного языка позволило значительно повысить качество решения таких задач, как машинный перевод, автоматическая классификация текстов, анализ тональности и генерация последовательностей. В отличие от традиционных статистических методов, нейросетевые модели способны учитывать контекст и формировать распределённые векторные представления слов (word embeddings), отражающие их семантические свойства.

Особенность текстовых данных заключается в наличии зависимостей между элементами последовательности. Полносвязные нейронные сети не способны эффективно учитывать порядок слов, поскольку обрабатывают входные данные как независимые признаки. Это ограничение привело к разработке специализированных архитектур, способных учитывать временную или последовательную структуру данных.

Таким образом, нейронные сети стали фундаментальным инструментом современного машинного обучения и обработки естественного языка. Их развитие привело к созданию рекуррентных архитектур, предназначенных для работы с последовательностями, что делает их особенно актуальными в рамках рассматриваемой темы.

2 РЕКУРРЕНТНЫЕ НЕЙРОННЫЕ СЕТИ

2.1 Архитектура RNN

Рекуррентные нейронные сети (Recurrent Neural Networks, RNN) представляют собой специализированный класс нейронных сетей, предназначенный для обработки последовательных данных [3]. В отличие от полносвязных (feedforward) сетей, рекуррентные сети обладают внутренним состоянием (hidden state) [4], которое передаётся от одного шага последовательности к другому. Это позволяет учитывать информацию о предыдущих элементах при обработке текущего входа.

Основной особенностью архитектуры RNN является наличие рекуррентной связи. На каждом шаге времени t сеть получает входной вектор x_t и предыдущее скрытое состояние h_{t-1} , формируя новое скрытое состояние h_t [3]:

$$h_t = f(W_x x_t + W_h h_{t-1} + b),$$

где:

- W_x — матрица весов входных связей,
- W_h — матрица рекуррентных весов,
- b — вектор смещения,
- f — функция активации (чаще всего используется $\tanh$ или ReLU).

Выход сети на каждом шаге может вычисляться следующим образом:

$$y_t = g(W_y h_t + c),$$

где W_y — матрица весов выходного слоя, c — вектор смещения выходного слоя, g — функция активации (например, softmax для задач классификации).

Таким образом, скрытое состояние h_t аккумулирует информацию обо всех предыдущих входах последовательности. Формально его можно представить как функцию от всех предыдущих элементов:

$$h_t = f(x_t, x_{t-1}, x_{t-2}, \dots, x_1).$$

Для наглядного понимания работы RNN используется представление сети в развёрнутом виде (unrolled form), при котором рекуррентная структура отображается как последовательность одинаковых блоков, соединённых во времени. При этом все временные шаги используют одни и те же параметры W_x , W_h и W_y , что существенно сокращает количество обучаемых параметров.

Обучение рекуррентной нейронной сети осуществляется методом обратного распространения ошибки во времени (Backpropagation Through Time, BPTT) [3]. Данный алгоритм представляет собой модификацию классического backpropagation и предполагает распространение градиента через все временные шаги последовательности. Функция потерь вычисляется как сумма ошибок на каждом шаге:

$$L = \sum_{t=1}^T L_t(y_t, \hat{y}_t),$$

где T — длина последовательности, L_t — функция потерь на шаге t , $\hat{y}_t$ — истинное значение.

Одной из ключевых особенностей RNN является способность моделировать временные зависимости различной длины. Это делает их эффективными в задачах анализа текста, распознавания речи, временных рядов и других последовательных данных [5]. Однако при увеличении длины последовательности обучение становится затруднительным из-за проблем с распространением градиента, что подробно рассматривается в следующем разделе.

С точки зрения структуры можно выделить несколько вариантов использования RNN:

- «один ко многим» (one-to-many) — генерация последовательности по одному входу;
- «многие к одному» (many-to-one) — классификация последовательности;
- «многие ко многим» (many-to-many) — преобразование одной последовательности в другую.

Такая гибкость архитектуры делает рекуррентные нейронные сети универсальным инструментом для работы с последовательными данными. Несмотря на ограничения классической архитектуры, именно RNN заложили основу для дальнейшего развития моделей глубокого обучения в области обработки естественного языка.

2.2 Проблема исчезающего градиента

Обучение рекуррентных нейронных сетей осуществляется методом обратного распространения ошибки во времени (Backpropagation Through Time, BPTT). При этом градиенты функции потерь вычисляются для каждого временного шага и затем суммируются для обновления весов сети [3]. Для сети с T временными шагами градиент функции потерь L по рекуррентным весам W_h можно записать как:

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^T \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial W_h}.$$

Так как скрытое состояние h_t зависит от всех предыдущих шагов $h_{t-1}, h_{t-2}, \dots, h_1$, вычисление градиента включает произведение большого числа матриц, соответствующих производным функций активации [1]. Для длинных последовательностей это приводит к двум основным проблемам: исчезающий градиент и взрывающийся градиент.

Исчезающий градиент возникает, когда нормы градиентов на каждом шаге меньше единицы, и произведение этих значений при распространении назад стремится к нулю. В результате веса сети, отвечающие за долгосрочные зависимости, практически не обновляются, и сеть не способна «запомнить» информацию о ранних элементах последовательности. Математически это можно выразить через произведение Якобианов:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}.$$

Если собственные значения матриц $\frac{\partial h_i}{\partial h_{i-1}}$ меньше единицы, произведение стремится к нулю при увеличении $t - k$, что и является причиной исчезновения градиента [3].

Взрывающийся градиент наблюдается, когда эти собственные значения больше единицы. Тогда градиент растёт экспоненциально, что приводит к нестабильному обучению и резким изменениям весов. Для борьбы с этой проблемой применяются методы нормализации градиентов (gradient clipping) [4], ограничивающие их максимальное значение.

Проблема исчезающего градиента особенно критична для задач, где необходимо учитывать долгосрочные зависимости, например:

- анализ сложных предложений в текстах, где связь между словами может простираться на десятки слов;
- машинный перевод, где корректная генерация слова зависит от контекста всего предложения;
- распознавание речи, где предыдущие сегменты аудио влияют на интерпретацию текущего.

Для наглядности проблему часто иллюстрируют графически: градиент по мере распространения назад уменьшается экспоненциально, и сигналы из ранних шагов не доходят до первых слоёв сети. Это ограничение классической RNN послужило стимулом для разработки архитектур, способных эффективно сохранять долгосрочную информацию, таких как LSTM и GRU, о которых будет рассказано в следующем разделе.

Таким образом, проблема исчезающего градиента является фундаментальным ограничением классических рекуррентных сетей и определяет необходимость усовершенствованных моделей, способных обрабатывать длинные последовательности без потери информации.

2.3 Модификации RNN: LSTM и GRU

Классические рекуррентные нейронные сети, как уже отмечалось, сталкиваются с проблемой исчезающего и взрывающегося градиента, что ограничивает их способность моделировать долгосрочные зависимости в последовательностях. Для преодоления этих ограничений были предложены усовершенствованные архитектуры RNN: Long Short-Term Memory (LSTM) и Gated Recurrent Unit (GRU) [3].

2.3.1 Архитектура LSTM

LSTM была предложена Хохрайтером и Шмидхубером в 1997 году и предназначена для сохранения информации на длительные промежутки времени [3]. Основным элементом LSTM — память (cell state) C_t , которая проходит через последовательность практически без изменений, подвергаясь лишь контролируемым модификациям с помощью механизма «врат» (gates).

В LSTM используются три основных типа ворот:

- **Входные ворота** i_t , определяющие, какая информация нового входа x_t должна быть добавлена в память:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i),$$

- **Забывающие ворота** f_t , решающие, какая часть предыдущей памяти C_{t-1} будет сохранена:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f),$$

- **Выходные ворота** o_t , определяющие, какая информация из памяти будет использоваться для формирования скрытого состояния h_t :

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o).$$

Обновление состояния памяти осуществляется следующим образом:

$$\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c),$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t,$$

$$h_t = o_t \odot \tanh(C_t),$$

где $\odot$ — поэлементное умножение, а σ — сигмоидальная функция. Благодаря механизму ворот LSTM способна сохранять и контролировать поток информации на протяжении длительных последовательностей, что делает её устойчивой к исчезающему градиенту.

2.3.2 Архитектура GRU

Gated Recurrent Unit (GRU) была предложена Чо и коллегами в 2014 году как более компактная альтернатива LSTM [3]. В GRU отсутствует отдельная память cell state; вместо этого используется комбинация скрытого состояния и двух ворот [4]:

- **Ворота обновления** z_t , контролирующие, какая часть скрытого состояния будет обновлена:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z),$$

- **Ворота сброса** r_t , определяющие, какая часть предыдущего скрытого состояния будет учтена при формировании нового состояния:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r).$$

Новое скрытое состояние вычисляется как:

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h),$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

GRU обладает меньшим количеством параметров по сравнению с LSTM, что ускоряет обучение и снижает требования к объёму данных, сохраняя при этом способность моделировать долгосрочные зависимости.

2.3.3 Сравнение LSTM и GRU

- LSTM имеет три ворота и отдельное состояние памяти, что делает её более гибкой, но несколько более тяжёлой для обучения.
- GRU объединяет некоторые функции ворот и использует одно скрытое состояние, что упрощает архитектуру и ускоряет обучение.
- Обе архитектуры позволяют эффективно решать проблему исчезающего градиента и моделировать долгосрочные зависимости.
- Выбор между LSTM и GRU зависит от конкретной задачи, объёма данных и требований к скорости обучения.

Таким образом, модификации RNN в виде LSTM и GRU обеспечивают возможность работы с длинными последовательностями, сохраняя информацию о предыдущих шагах и преодолевая ограничения классических рекуррентных сетей. Эти архитектуры широко применяются в задачах обработки естественного языка, включая машинный перевод, генерацию текста, анализ тональности и классификацию документов.

3 ПРИМЕНЕНИЕ RNN В ЗАДАЧАХ NLP

3.1 Классификация текстов

Классификация текстов является одной из наиболее распространённых задач обработки естественного языка. Она заключается в автоматическом определении категории, к которой принадлежит текстовый документ, исходя из его содержания [2]. Примеры включают классификацию новостных статей по темам, определение тональности отзывов, фильтрацию спама и определение жанра текста.

Рекуррентные нейронные сети особенно хорошо подходят для задач классификации текстов благодаря своей способности учитывать последовательную структуру данных. В отличие от полносвязных сетей, которые рассматривают слова как независимые признаки, RNN анализируют текст как последовательность токенов, учитывая порядок слов и контекст. Это позволяет моделировать как локальные, так и долгосрочные зависимости в тексте.

3.1.1 Архитектура RNN для классификации

На практике для классификации текста часто используется архитектура «многие к одному» (many-to-one), где входной последовательностью является текст, а выходом — категория документа. Процесс обработки выглядит следующим образом:

1. **Векторизация текста:** каждое слово преобразуется в числовой вектор с помощью one-hot encoding или более современных методов, таких как word embeddings (Word2Vec, GloVe, FastText) [3].
2. **Обработка последовательности RNN:** последовательность векторов подаётся на вход RNN, которая формирует скрытые состояния h_t для каждого токена, аккумулируя информацию о предыдущих словах.
3. **Получение итогового представления:** скрытое состояние последнего шага h_T используется как обобщённое представление всей последовательности текста.
4. **Классификация:** итоговое скрытое состояние подаётся на полносвязный слой с функцией активации softmax для получения вероятностей принадлежности текста к каждой категории:

$$\hat{y} = \text{softmax}(W_c h_T + b_c),$$

где W_c и b_c — параметры выходного слоя.

3.1.2 Преимущества использования RNN

Использование RNN в классификации текстов имеет ряд преимуществ:

- **Учет контекста:** RNN способны учитывать порядок слов и контекст, что особенно важно для длинных предложений и сложных текстов.
- **Моделирование зависимостей:** RNN и их модификации (LSTM, GRU) позволяют эффективно моделировать долгосрочные зависимости, что улучшает точность классификации.
- **Гибкость:** архитектура «многие к одному» может быть адаптирована для различных типов текстов и задач классификации.

3.1.3 Современные подходы

Современные подходы к классификации текстов часто используют комбинацию RNN с другими архитектурами:

- **Bi-directional RNN (BiRNN):** сеть обрабатывает последовательность в прямом и обратном направлении, что позволяет учитывать как предыдущий, так и последующий контекст каждого слова [5].
- **RNN + Attention:** механизм внимания позволяет сети сосредотачиваться на наиболее значимых словах текста, повышая качество классификации.
- **RNN + CNN:** комбинация сверточных и рекуррентных слоёв используется для извлечения локальных признаков и последовательных зависимостей одновременно.

3.1.4 Применение на практике

RNN и их модификации широко применяются для решения практических задач классификации текста:

- Определение тональности отзывов пользователей (положительный/отрицательный отзыв).
- Тематическая классификация новостных статей и блогов.
- Классификация писем и сообщений (например, фильтрация спама).
- Анализ социальных сетей для выявления трендов и эмоциональной окраски публикаций.

Таким образом, рекуррентные нейронные сети обеспечивают эффективную обработку текстов с учётом последовательной структуры и контекста, что делает их незаменимым инструментом для современных систем классификации текста.

3.2 Машинный перевод

Машинный перевод (Machine Translation, MT) является одной из ключевых задач обработки естественного языка, направленной на автоматическое преобразование текста с одного языка на другой [6]. Классические статистические методы перевода уступили место нейросетевым моделям, которые обеспечивают более высокое качество и естественность перевода.

Рекуррентные нейронные сети стали фундаментальной основой современных систем машинного перевода благодаря своей способности моделировать последовательности и учитывать контекст. Основной архитектурой является модель Encoder-Decoder, где одна RNN кодирует входной текст, а другая генерирует перевод [6].

3.2.1 Архитектура Encoder-Decoder

Архитектура Encoder-Decoder состоит из двух основных компонентов:

1. **Encoder:** RNN (или LSTM/GRU) принимает последовательность входных слов на исходном языке и преобразует её в компактное представление — скрытое состояние h_T , которое содержит информацию о всей последовательности:

$$h_t^{\text{enc}} = f(W_x x_t + W_h h_{t-1} + b),$$

где x_t — векторное представление входного слова, а h_t^{enc} — скрытое состояние кодировщика на шаге t .

2. **Decoder:** RNN принимает скрытое состояние кодировщика и генерирует последовательность слов на целевом языке. На каждом шаге t декодер предсказывает слово y_t на основе предыдущего состояния h_{t-1}^{dec} и предыдущего сгенерированного слова y_{t-1} :

$$h_t^{\text{dec}} = f(W_x y_{t-1} + W_h h_{t-1}^{\text{dec}} + W_c h_T^{\text{enc}} + b),$$

$$y_t = \text{softmax}(W_y h_t^{\text{dec}} + c).$$

На практике для улучшения качества перевода используют следующие методы:

3.2.2 Двухнаправленные кодировщики (Bi-directional Encoder)

Вместо обычной однонаправленной RNN используют BiRNN для кодировщика, которая обрабатывает последовательность в прямом и обратном направлении. Это позволяет учитывать как предыдущий, так и последующий контекст слова [1]:

$$h_t^{\text{enc}} = [\vec{h}_t, \overleftarrow{h}_t]$$

3.2.3 Механизм внимания (Attention)

Механизм внимания позволяет декодеру «фокусироваться» на наиболее значимых словах исходного текста при генерации каждого слова перевода. Вместо передачи единственного скрытого состояния h_T^{enc} используется взвешенная комбинация всех скрытых состояний кодировщика:

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_i^{\text{enc}},$$

где коэффициенты $\alpha_{t,i}$ отражают степень важности каждого входного токена для генерации y_t . Выход декодера тогда зависит от текущего состояния h_t^{dec} и контекстного вектора c_t :

$$y_t = \text{softmax}(W_y[h_t^{\text{dec}}; c_t] + b)$$

Использование внимания значительно повышает качество перевода, особенно для длинных предложений, так как декодер получает доступ ко всем входным токенам, а не только к последнему скрытому состоянию кодировщика [6].

3.2.4 Практические применения

RNN и их модификации применяются в реальных системах машинного перевода, таких как:

- Онлайн-переводчики (например, Google Translate, Yandex Translate) для перевода текстов и документов.
- Системы локализации программного обеспечения и веб-контента.
- Перевод субтитров и автоматическая генерация многоязычных видео.
- Поддержка двуязычных чат-ботов и систем автоматического ответа.

3.2.5 Преимущества RNN в машинном переводе

- Эффективное моделирование последовательностей различной длины.
- Возможность учитывать долгосрочные зависимости между словами и фразами.
- Совместимость с механизмом внимания, что повышает точность перевода длинных и сложных предложений.
- Гибкость для интеграции с другими архитектурами (например, Convolutional Neural Networks или Transformer-подходы для гибридных моделей).

Таким образом, архитектуры RNN Encoder-Decoder с механизмом внимания обеспечивают высокое качество машинного перевода и являются ключевыми компонентами современных систем NLP.

3.3 Генерация текста

Генерация текста представляет собой задачу создания последовательности слов, фраз или предложений на основе заданного контекста [3]. В отличие от машинного перевода, где исходный текст задаёт семантическую основу перевода, генерация текста часто предполагает частичное или полное «придумывание» содержания, что делает задачу более творческой и сложной.

3.3.1 Использование RNN для генерации текста

Рекуррентные нейронные сети являются естественным инструментом для генерации текстов, так как они способны моделировать последовательные зависимости между словами. Процесс генерации обычно состоит из следующих шагов:

1. **Инициализация:** задаётся начальное слово или последовательность слов $x_1, \dots, x_k$, которые служат начальным контекстом.
2. **Пошаговая генерация:** на каждом временном шаге RNN предсказывает вероятности следующего слова x_{t+1} на основе текущего скрытого состояния h_t и предыдущего слова x_t :
$$h_t = f(W_x x_t + W_h h_{t-1} + b), \quad y_t = \text{softmax}(W_y h_t + c),$$
где y_t — вектор вероятностей для словарного запаса.
3. **Выбор слова:** следующее слово выбирается на основе распределения y_t (например, с помощью жадного выбора, выборки по вероятностям или с применением температуры для управления разнообразием текста).
4. **Повторение процесса:** сгенерированное слово подаётся на вход сети для следующего шага, и процесс продолжается до достижения заданной длины текста или сигнала конца последовательности.

3.3.2 Роль LSTM и GRU

Использование LSTM или GRU значительно улучшает качество генерации текста [3]. Эти архитектуры позволяют:

- **Сохранять долгосрочные зависимости:** важные слова и контексты, появившиеся в начале текста, учитываются при генерации последующих слов.

- **Предотвращать исчезающий градиент:** что обеспечивает обучение на больших корпусах текстов и создание более связного текста.
- **Гибко управлять информацией:** с помощью ворот (input, forget, output) в LSTM или update/reset в GRU можно контролировать поток информации, избегая «забывания» важного контекста.

3.3.3 Стратегии генерации текста

При генерации текста применяются различные стратегии выбора следующего слова:

- **Greedy search (жадный поиск):** выбирается слово с наибольшей вероятностью на каждом шаге. Простой метод, но может приводить к повторению фраз и предсказуемости текста.
- **Sampling (выборка):** слово выбирается случайным образом с учётом вероятностного распределения, что обеспечивает разнообразие текста.
- **Beam search:** рассматривается несколько наиболее вероятных последовательностей одновременно и выбирается наиболее оптимальная комбинация. Часто используется в генерации текста для достижения баланса между качеством и разнообразием.
- **Temperature scaling:** изменение «температуры» распределения вероятностей позволяет регулировать креативность сети — низкая температура делает текст более предсказуемым, высокая — более разнообразным и неожиданным.

3.3.4 Применение генерации текста

Генерация текста с помощью RNN и их модификаций применяется в широком спектре задач NLP:

- **Чат-боты и виртуальные ассистенты:** автоматическое формирование ответов на вопросы пользователей.
- **Автоматическое написание статей и обзоров:** генерация текстов на основе заданной темы или структуры.
- **Создание художественных текстов:** генерация стихов, рассказов и сценариев.
- **Дополнение текста и автозаполнение:** предсказание следующего слова или предложения для ускорения работы пользователей в текстовых редакторах.

3.3.5 Преимущества и ограничения

Преимущества:

- Возможность генерации связного текста с учётом контекста.
- Устойчивость к длинным последовательностям при использовании LSTM/GRU.
- Гибкость для интеграции с другими методами NLP, например, embeddings или attention.

Ограничения:

- Могут возникать ошибки при генерации длинных текстов без дополнительного контекста.
- Возможна потеря когерентности при генерации очень больших последовательностей.
- Требуется большой объём обучающих данных для достижения реалистичности и разнообразия текста.

Таким образом, рекуррентные нейронные сети, а особенно их модификации LSTM и GRU, являются эффективным инструментом генерации текста. Они позволяют создавать тексты, учитывая контекст и структурные зависимости, что делает их важными компонентами современных систем NLP.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были исследованы теоретические и практические аспекты применения рекуррентных нейронных сетей в обработке естественного языка. Основные выводы можно сформулировать следующим образом:

1. **Актуальность темы:** Обработка естественного языка остаётся одной из наиболее востребованных областей искусственного интеллекта. Рекуррентные нейронные сети (RNN) и их модификации обеспечивают эффективное моделирование последовательностей, что делает их ключевым инструментом для анализа и генерации текстов.
2. **Теоретическая база:** Были рассмотрены архитектурные особенности RNN, включая скрытое состояние и рекуррентные связи, а также алгоритмы обучения с использованием обратного распространения ошибки во времени (BPTT). Выявлены ограничения классических RNN, связанные с проблемой исчезающего и взрывающегося градиента при обработке длинных последовательностей.
3. **Модификации RNN:** LSTM и GRU представляют собой усовершенствованные архитектуры, способные эффективно сохранять долгосрочную информацию и преодолевать ограничения классических RNN. LSTM использует отдельное состояние памяти и три типа ворот, а GRU — более компактную структуру с двумя воротами, обеспечивая баланс между качеством моделирования и вычислительной эффективностью.
4. **Применение в NLP:** Рекуррентные нейронные сети успешно применяются в задачах:
 - классификации текстов, включая анализ тональности, тематическую классификацию и фильтрацию спама;
 - машинного перевода, используя архитектуру Encoder-Decoder с механизмом внимания;
 - генерации текста, включая чат-ботов, автозаполнение, создание статей и художественных текстов.
5. **Преимущества использования RNN:** RNN и их модификации позволяют учитывать порядок слов и контекст, моделировать долгосрочные зависимости, интегрироваться с современными методами NLP (embeddings, attention, BiRNN) и обеспечивать гибкость при решении различных задач.

6. Ограничения и рекомендации: Несмотря на высокую эффективность, RNN имеют ограничения при генерации очень длинных последовательностей и требуют значительных объёмов обучающих данных. Для повышения качества решений рекомендуется:

- использовать механизмы внимания и двунаправленные архитектуры;
- комбинировать RNN с другими нейросетевыми подходами, такими как CNN или Transformer;
- применять методы регуляризации и нормализации градиентов для стабильного обучения.

7. Практическая значимость: Полученные знания о работе RNN, LSTM и GRU могут быть использованы при разработке интеллектуальных систем анализа текста, включая автоматические переводчики, системы генерации текстов и классификаторы больших текстовых корпусов.

Таким образом, проведённое исследование подтверждает высокую эффективность рекуррентных нейронных сетей в задачах обработки естественного языка и демонстрирует их потенциал для дальнейшего применения в различных областях искусственного интеллекта и информационных технологий.

Список использованных источников

1. Ахтямов А. И., Ахтямова Н. Р. Разработка вопросно-ответной системы на основе алгоритмов машинного обучения // Труды IV Международной научной конференции «Инновационные технологии в науке и образовании». – 2017. – С. 62–65.
2. Платонов Е. В. Методы и алгоритмы обработки естественного языка в вопросно-ответных системах // Вестник компьютерных и информационных технологий. – 2018. – № 2(16). – С. 84–90.
3. Гольдберг Й. Нейросетевые методы в обработке естественного языка / пер. с англ. А. А. Слинкина. – М.: ДМК Пресс, 2019. – 282 с.: ил.
4. Азизова Н. Нейронные сети и обработка естественного языка (NLP): простое объяснение для новичков [Электронный ресурс] // Tehstd.ru. – 2025. – 17 июля. – Режим доступа: <https://tehstd.ru/news/neyronnye-seti-i-obrabotka-estestvennogo-yazyka-nlp-prostoe-obyasnenie-dlya-novichkov/> (дата обращения: 03.03.2026).
5. Лекция 11. Машинное обучение в обработке естественного языка [Электронный ресурс] / сост. – Режим доступа: https://zhanibekov.edu.kz/media/university/faculties/materials/%D0%9B%D0%B5%D0%BA%D1%86%D0%B8%D1%8F_11_1.pdf (дата обращения: 03.03.2026).
6. Бурнашев Р. Ф., Анварова Л. А. Применение нейронных сетей в автоматическом переводе и обработке естественного языка [Электронный ресурс] // Universum: технические науки. – 2024. – № 4(121). – Режим доступа: <https://7universum.com/ru/tech/archive/item/17192> (дата обращения: 03.03.2026).